

Práctica 3 — Implementación Boyer-Moore

Diseño de Algoritmos • Trabajo en grupos

Repaso activo

Antes de programar, piensa

- ¿Qué hacía la heurística **bad character** cuando el carácter del texto no estaba en el patrón?
- ¿En qué momento entra a jugar la heurística **good suffix**?
- ¿Cómo se combinan ambos saltos para decidir cuánto avanzar?

Preguntas guía — construcción del algoritmo

Reflexiona mientras construimos el pseudocódigo

- Boyer-Moore compara el patrón en una dirección particular. **¿En qué posición del patrón comienza la comparación, y por qué tiene sentido empezar justo ahí?**
- Cuando ocurre un mismatch en una posición j del patrón, **¿qué información necesitas calcular antes de decidir cuánto vas a desplazar el patrón sobre el texto?**
- El recorrido no termina solo cuando hay un mismatch. **¿Qué condición sobre j indica que el patrón completo coincidió con el texto?**

Esqueleto de trabajo

En la pizarra se anotarán únicamente las **declaraciones de las funciones**. Ustedes deben completar la implementación dentro del esqueleto que aparece a continuación.

```
#define ALFABETO 256

void construir_bad_char(char *P, int m, int bad_char[ALFABETO]) {
    // To do
}

void construir_good_suffix(char *P, int m, int gs[]) {
    // To do
}

int buscar_boyer_moore(char *T, char *P) {
    // To do
    return -1;
}

int main() {
    char *textos[2] = {
        "comocomotecomotecomotecomotecomotecomoto",
        "trestigrestristerrostrigantristetigretragaba"
    };
}
```

```
char *patrones[4] = {
    "tecomo",
    "triste",
    "comote",
    "zzzz"
};

int t, p;

for (t = 0; t < 2; t++) {
    for (p = 0; p < 4; p++) {
        printf("TEXT0: %s\n", textos[t]);
        printf("PATRON: %s\n", patrones[p]);

        int resultado = buscar_boyer_moore(textos[t], patrones[p]);

        if (resultado != -1) {
            printf("Encontrado en la posicion %d\n", resultado);
        } else {
            printf("No encontrado\n");
        }

        printf("\n");
    }
}

return 0;
}
```

Trabajo en grupo

Instrucciones

- Formen grupos de **2 a 3 personas**.
- Completen en conjunto las funciones `construir_bad_char`, `construir_good_suffix` y `buscar_boyer_moore` dentro del esqueleto.
- Verifiquen que el programa compila y que las salidas coinciden con lo esperado para los 4 patrones sobre los 2 textos.
- Tienen **45 minutos** para implementar y comprobar el funcionamiento.

Recuerden

No se espera una implementación perfecta ni optimizada. El objetivo es que la lógica de ambas heurísticas quede correctamente reflejada en el código y que el algoritmo entregue resultados coherentes.